

```
#!/usr/bin/env python3
```

```
"""
```

```
doc_to_skill_and_tool.py
```

```
-----
```

PURPOSE FOR BEGINNER DEVELOPERS:

This script automates the "Dual Extraction Mandate" from the Definitive Master Spec.

It takes any raw markdown or text document, inspects it for:

1. Code fences (``bash, ``sh, ``python) -> Extracts them as standalone, executable scripts.
2. Conceptual rules & headers -> Formats them as an agent SKILL.md with YAML frontmatter.
3. Automatically computes SHA256 hashes and registers everything in manifest.jsonl.

```
"""
```

```
import os
```

```
import re
```

```
import json
```

```
import hashlib
```

```
from pathlib import Path
```

```
from datetime import datetime, timezone
```

```
WORKSPACE_ROOT = Path(os.environ.get("VAULT_ROOT", "~/novae-xorpus")).expanduser()
```

```
SKILLS_DIR = WORKSPACE_ROOT / "04_skills_runtime" / "prompt_skills"
```

```
TOOLS_CLI_DIR = WORKSPACE_ROOT / "04_skills_runtime" / "extracted_tools" / "cli"
```

```
TOOLS_PY_DIR = WORKSPACE_ROOT / "04_skills_runtime" / "extracted_tools" / "wrappers"
```

```
MANIFEST_FILE = WORKSPACE_ROOT / "04_skills_runtime" / "manifest.jsonl"
```

```
def sha256_file(filepath: Path) -> str:
```

```
    h = hashlib.sha256()
```

```
    with open(filepath, "rb") as f:
```

```
        while chunk := f.read(65536):
```

```
            h.update(chunk)
```

```
    return h.hexdigest()[:16]
```

```
def extract_tools_and_skills(source_file: Path):
```

```
    print(f"[*] Processing: {source_file.name}")
```

```
    content = source_file.read_text(encoding="utf-8", errors="ignore")
```

```
    base_name = source_file.stem.lower().replace(" ", "_").replace("-", "_")
```

```
    SKILLS_DIR.mkdir(parents=True, exist_ok=True)
```

```
    TOOLS_CLI_DIR.mkdir(parents=True, exist_ok=True)
```

```
    TOOLS_PY_DIR.mkdir(parents=True, exist_ok=True)
```

```
    extracted_artifacts = []
```

```
code_pattern = re.compile(r"```(bash|sh|python)\n(.*)```", re.DOTALL)
matches = code_pattern.findall(content)
```

```
for idx, (lang, code) in enumerate(matches):
```

```
    code_stripped = code.strip()
```

```
    if len(code_stripped) < 20:
```

```
        continue
```

```
    if lang in ["bash", "sh"]:
```

```
        out_file = TOOLS_CLI_DIR / f"{base_name}_tool_{idx}.sh"
```

```
        if not code_stripped.startswith("#!"):
```

```
            code_stripped = f"#!/usr/bin/env bash\nset -euo pipefail\n\n{code_stripped}"
```

```
        out_file.write_text(code_stripped + "\n", encoding="utf-8")
```

```
        out_file.chmod(0o755)
```

```
        tool_type = "cli_shell"
```

```
    else:
```

```
        out_file = TOOLS_PY_DIR / f"{base_name}_tool_{idx}.py"
```

```
        if not code_stripped.startswith("#!"):
```

```
            code_stripped = f"#!/usr/bin/env python3\n\n{code_stripped}"
```

```
        out_file.write_text(code_stripped + "\n", encoding="utf-8")
```

```
        out_file.chmod(0o755)
```

```
        tool_type = "python_wrapper"
```

```
print(f" [+] Extracted Tool: {out_file.relative_to(WORKSPACE_ROOT)}")
```

```
extracted_artifacts.append({
```

```
    "type": "tool",
```

```
    "subtype": tool_type,
```

```
    "path": str(out_file.relative_to(WORKSPACE_ROOT)),
```

```
    "hash": sha256_file(out_file)
```

```
})
```

```
skill_file = SKILLS_DIR / f"{base_name}_skill.md"
```

```
yaml_header = f"```---
```

```
name: {base_name}
```

```
description: Procedural skill extracted from {source_file.name}.
```

```
extracted_at: {datetime.now(timezone.utc).isoformat()}
```

```
source_doc: {str(source_file.relative_to(WORKSPACE_ROOT))}
```

```
extracted_tools: {[a["path"] for a in extracted_artifacts]}
```

```
---
```

```
# Procedural Directives: {source_file.stem}
```

```
## Context & Objectives
```

This skill governs how agents interact with the tools extracted from `{source\_file.name}`.

Core Directives

{content[:800]} ... (compacted)

"""

```
skill_file.write_text(yaml_header, encoding="utf-8")
print(f" [+] Extracted Skill: {skill_file.relative_to(WORKSPACE_ROOT)}")
```

```
manifest_entry = {
    "id": f"EXT_{sha256_file(source_file)}",
    "source_doc": str(source_file.relative_to(WORKSPACE_ROOT)),
    "skill_created": str(skill_file.relative_to(WORKSPACE_ROOT)),
    "tools_created": extracted_artifacts,
    "timestamp": datetime.now(timezone.utc).isoformat()
}
```

```
with open(MANIFEST_FILE, "a", encoding="utf-8") as f:
    f.write(json.dumps(manifest_entry) + "\n")
print(f" [✓] Registered in {MANIFEST_FILE.name}\n")
```

```
if __name__ == "__main__":
```

```
    import sys
```

```
    if len(sys.argv) > 1:
```

```
        extract_tools_and_skills(Path(sys.argv[1]))
```

```
    else:
```

```
        print("Usage: python3 doc_to_skill_and_tool.py <path_to_source_file.md>")
```